



第二章 运算方法和运算部件



- ◆ 定点数加减运算
- ◆ 定点数乘法运算
- ◆ 定点数除法运算
- ◆ 硬件控制语言HCL



计算机中常用的数据表示格式有两种：

- **定点格式**——容许的数值范围有限，但要求的处理硬件比较简单。
- **浮点格式**——容许的数值范围很大，但要求的处理硬件比较复杂。

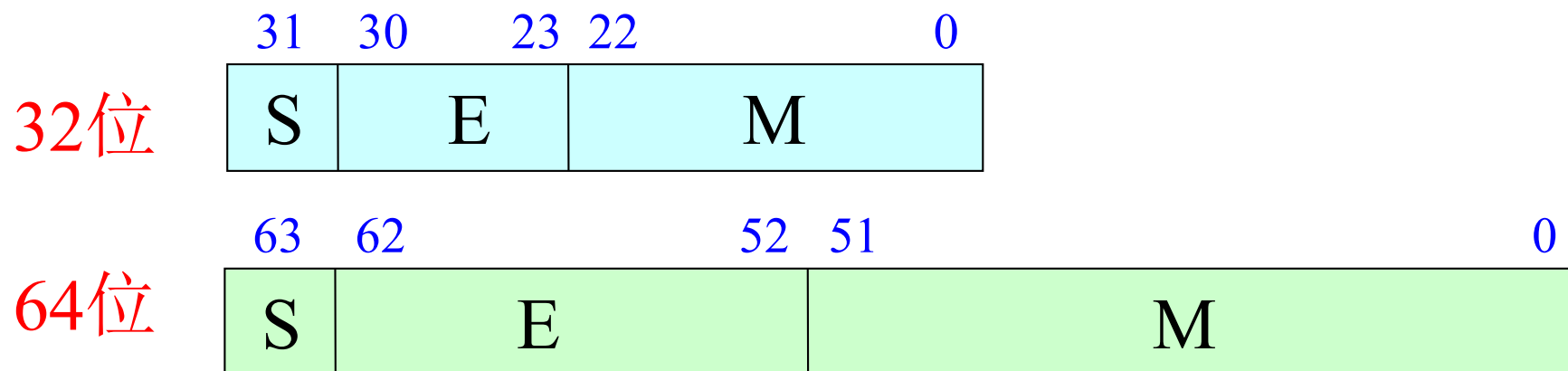
定点数的表示方法

定点表示：约定机器中所有数据的小数点位置是固定不变的。
(由于约定在固定的位置，小数点就不再使用记号“.”来表示。)

通常将数据表示成**纯小数**或**纯整数**。



按照 IEEE754 的标准，32位浮点数和64位浮点数的标准格式为：



S —— 尾数符号，0正1负；

M —— 尾数，纯小数表示，小数点放在尾数域的最前面。

采用原码表示。

E —— 阶码，采用“移码”表示；

阶符采用隐含方式，即采用“移码”方法来表示正负指数



1位加法器

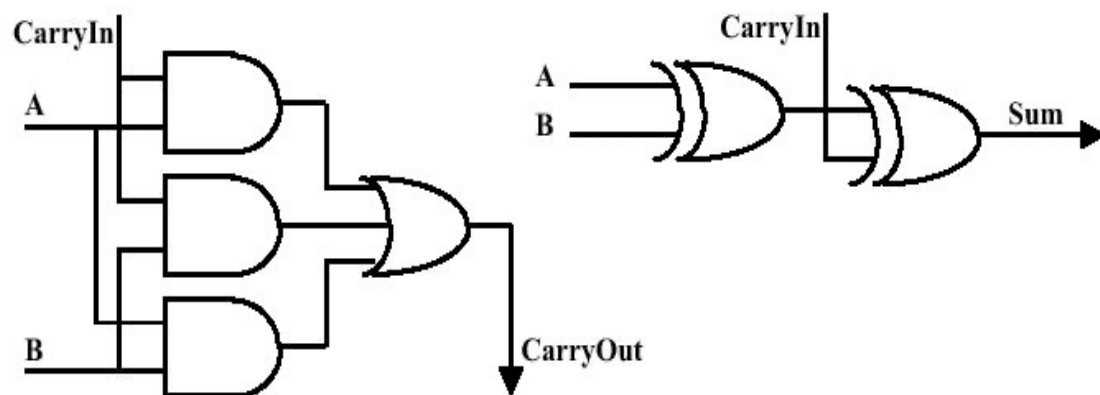
一位带进位的加法器称为全加器

– 两个加数为A和B，低位进位为Cin，
和为F，向高位的进位为Cout

化简后，逻辑表达式如下

$$F = A \oplus B \oplus C_{in}$$

$$C_{out} = A \& B \mid A \& C_{in} \mid B \& C_{in}$$

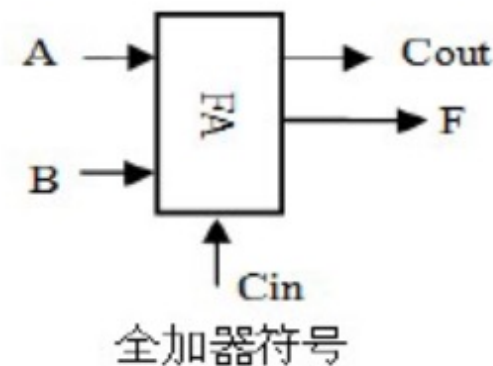


假定与/或门延迟为1ty，异或门3ty，则 “和” 与 “进位” 的延迟为多少？

和F的延迟为6ty；进位Cout延迟为2ty。

A	B	Cin	F	Cout
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

全加器真值表

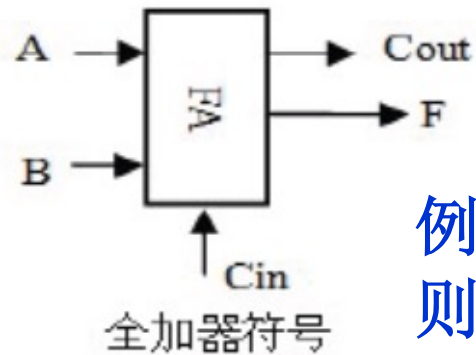


全加器符号

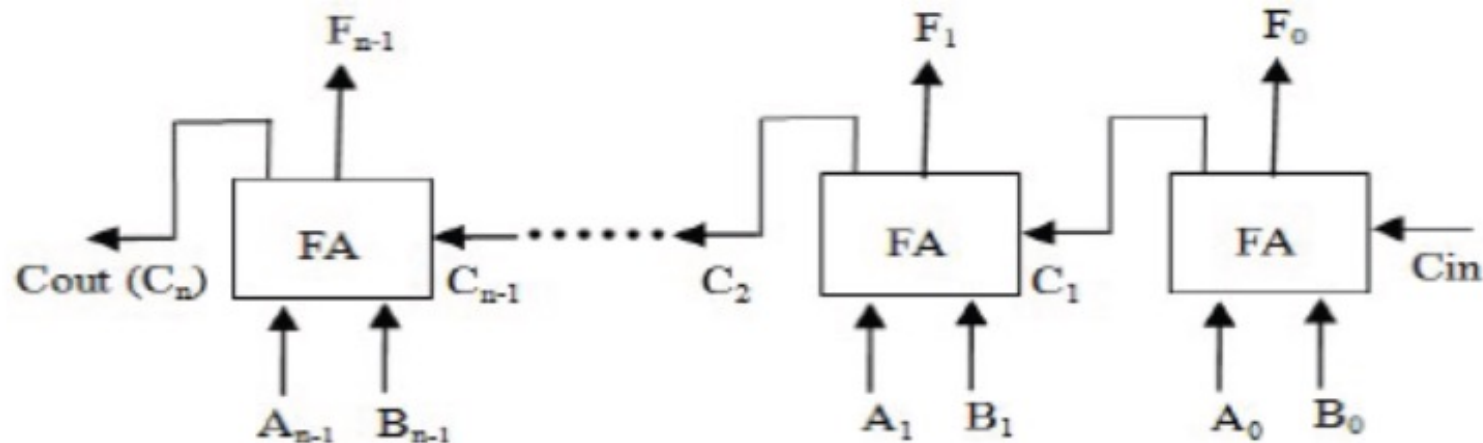
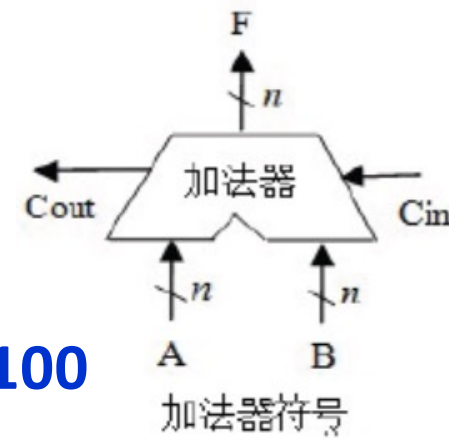


n位行波进位加法器

n位加法器可用n个全加器实现



例: $n=4$, $A=1001$, $B=1100$
则: $F=0101$, $Cout=1$





整数加/减运算器

计算机组成原理

• 补码加减运算公式

$$[A+B]_{\text{补}} = [A]_{\text{补}} + [B]_{\text{补}} \pmod{2^n}$$

$$[A-B]_{\text{补}} = [A]_{\text{补}} + [-B]_{\text{补}} \pmod{2^n}$$

— 实现减法的主要工作在于：求 $[-B]_{\text{补}}$

$$[-B]_{\text{补}} = [[B]_{\text{补}}]_{\text{反}} + 1$$

- 利用带标志加法器，可构造整数加/减运算器，进行以下运算：

无符号整数加、无符号整数减

带符号整数加、带符号整数减

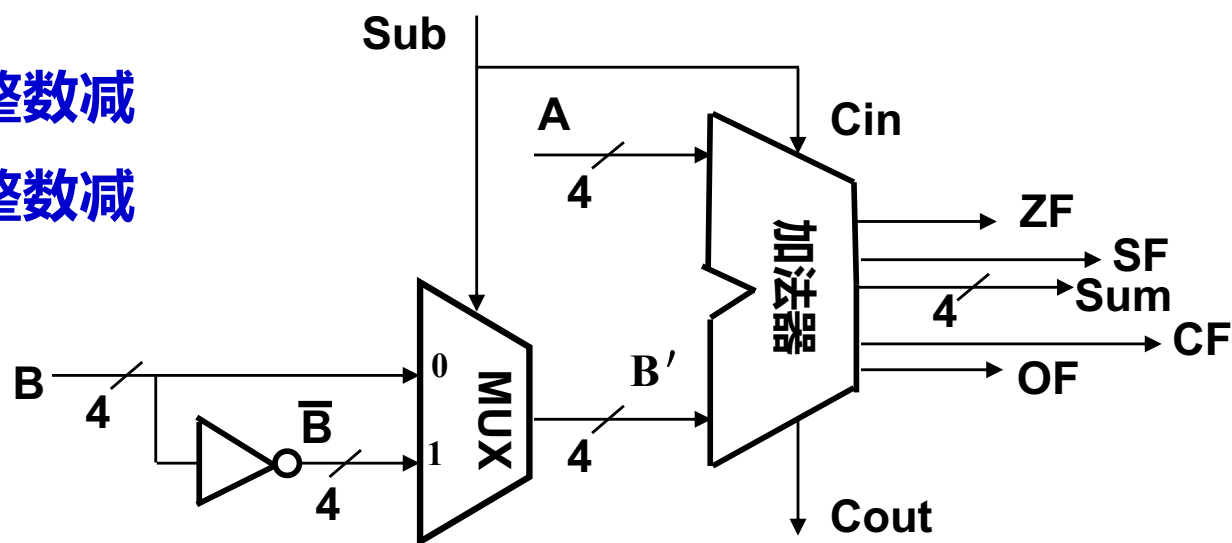
在整数加/减运算部件基础上，加上寄存器、移位器以及控制逻辑，就可实现**ALU**、**乘/除**运算以及**浮点**运算电路

有符号溢出标志**OF**：

① $OF = C_n \oplus C_{n-1}$

② 若A与B'同号但与Sum不同号，则1；否则0。

当Sub为1时，做减法
当Sub为0时，做加法



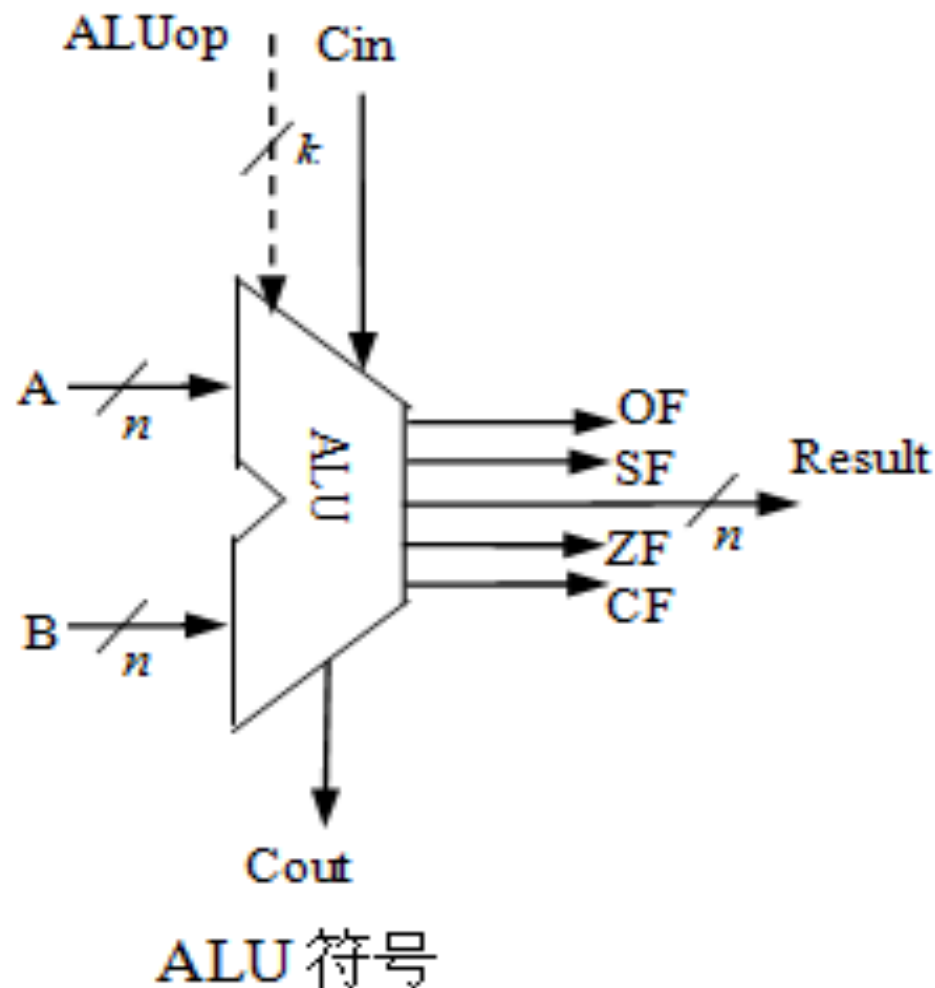
整数加/减运算部件



算术逻辑部件 (ALU)

计算机组成原理

- 进行**基本**算术运算与逻辑运算
 - 无符号整数加、减
 - 带符号整数加、减
 - 与、或、非、异或等逻辑运算
- 核心电路是**整数加/减运算部件**
- 输出**和/差等**，以及**标志信息**
- 有一个**操作控制端** (ALUop)，用来决定ALU所执行的处理功能。ALUop的位数k决定了操作的种类例如，当位数k为3时，ALU最多只有 $2^3=8$ 种操作。



ALUop	Result	ALUop	Result	ALUop	Result	ALUop	Result
0 0 0	A加B	0 1 0	A与B	1 0 0	A取反	1 1 0	A
0 0 1	A减B	0 1 1	A或B	1 0 1	$A \oplus B$	1 1 1	未用



- 为什么用先行进位方式?

串行进位加法器采用串行逐级传递进位，电路延迟与位数成正比关系。
因此，现代计算机采用一种先行进位(Carry look ahead)方式。

- 如何产生先行进位?

定义辅助函数： $G_i = A_i B_i$...进位生成函数

$P_i = A_i + B_i$...进位传递函数

通常把实现上述逻辑的电路称为进位生成/传递部件

全加逻辑方程： $S_i = A_i \oplus B_i \oplus C_i$ $C_{i+1} = G_i + P_i C_i$ ($i=0, 1, \dots, n$)

设 $n=4$,则： $C_1 = G_0 + P_0 C_0$

$$C_2 = G_1 + P_1 C_1 = G_1 + P_1 G_0 + P_1 P_0 C_0$$

$$C_3 = G_2 + P_2 C_2 = G_2 + P_2 G_1 + P_2 P_1 G_0 + P_2 P_1 P_0 C_0$$

$$C_4 = G_3 + P_3 C_3 = G_3 + P_3 G_2 + P_3 P_2 G_1 + P_3 P_2 P_1 G_0 + P_3 P_2 P_1 P_0 C_0$$

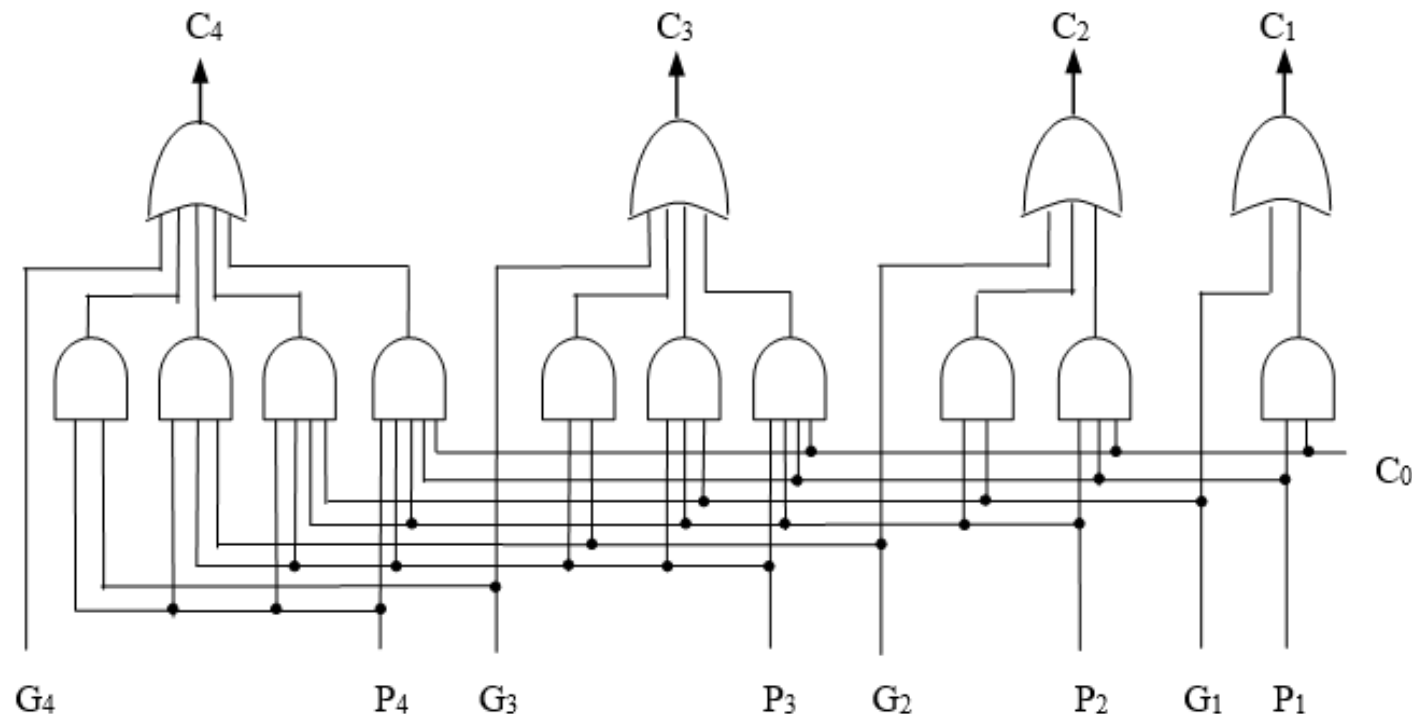
由上式可知:各进位之间无等待，相互独立并同时产生。

通常把实现上述逻辑的电路称为4位先行进位部件 (4位CLU)



CLA加法器

所有进位独立并同时生成



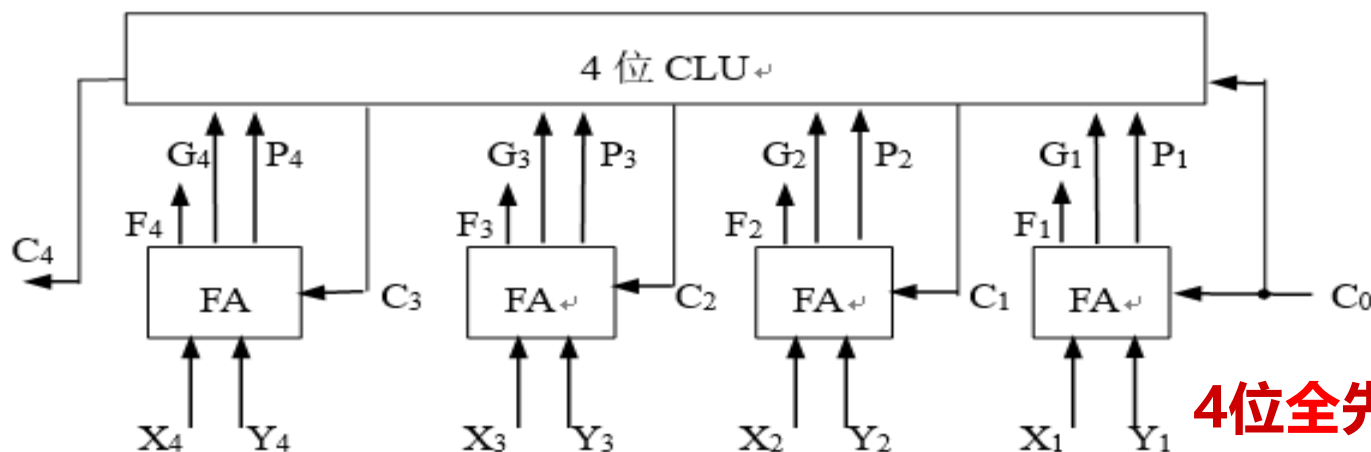
$$C_1 = G_1 + P_1 C_0$$

$$C_2 = G_2 + P_2 C_1 = G_2 + P_2 G_1 + P_2 P_1 C_0$$

$$C_3 = G_3 + P_3 C_2 = G_3 + P_3 G_2 + P_3 P_2 G_1 + P_3 P_2 P_1 C_0$$

$$C_4 = G_4 + P_4 C_3 = G_4 + P_4 G_3 + P_4 P_3 G_2 + P_4 P_3 P_2 G_1 + P_4 P_3 P_2 P_1 C_0$$

4位CLU



$$G_i = X_i Y_i$$

$$P_i = X_i + Y_i \text{ (或 } P_i = X_i \oplus Y_i \text{)}$$

$$F_i = X_i \oplus Y_i \oplus C_{i-1}$$

4位全先行进位加法器CLA



乘法运算

问题：已知 $[x]_{\text{补}}$ 和 $[y]_{\text{补}}$ ，求 $[x*y]_{\text{补}}$

因为 $[x*y]_{\text{补}} \neq [x]_{\text{补}} * [y]_{\text{补}}$ ，故不能直接用无符号整数乘法计算！

例如，若 $x=11$ ， $y=12$ ，求 $[x*y]_{\text{无}}=?$

$$[11]_{\text{无}} = 1011, [12]_{\text{无}} = 1100$$

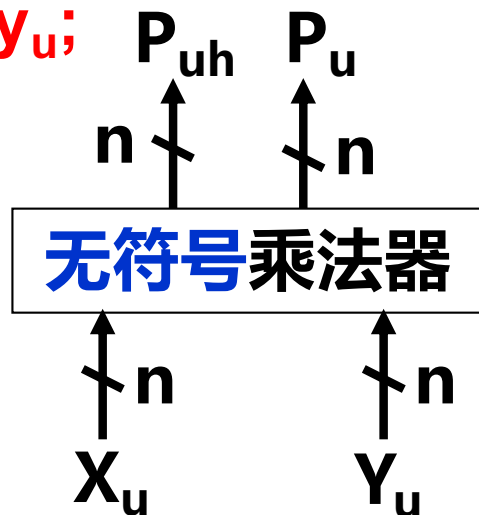
在无符号数乘法器中运算： $[11]_{\text{无}} * [12]_{\text{无}} = 1000 \ 0100$

例如，若 $x=-5$ ， $y=-4$ ，求 $[x*y]_{\text{补}}=?$

$$[-5]_{\text{补}} = 1011, [-4]_{\text{补}} = 1100$$

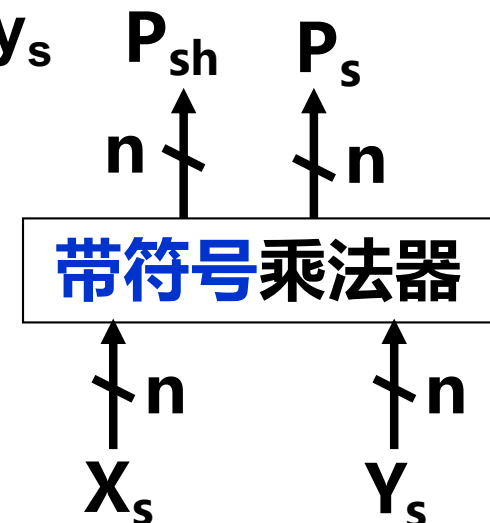
在补码乘法器中运算： $[x*y]_{\text{补}} = 1011 * 1100 = 0001 \ 0100$

$$p_u = x_u * y_u;$$



$$p_s = x_s * y_s$$

$$P_{uh} \neq P_{sh}$$





无符号数的乘法运算

假定: $X=x_0.x_1...x_n$, $Y=y_0.y_1...y_n$, 求 $x \times Y$

数值部分 $z_1...z_{2n} = (0.x_1...x_n) \times (0.y_1...y_n)$

(小数点位置约定, 不区分小数还是整数)

- Paper and pencil example: $n=4$

Multiplicand 0.1000

Multiplier $\times 0.1001$

0.00001000

0.00000000

0.00000000

0.010000

Product 0.01001000

$$X \times Y = \sum_{i=1}^4 (X \times y_i \times 2^{n-i} \times 2^{-n})$$

$$X \times y_4 \times 2^{4-4} \times 2^{-4}$$

$$X \times y_3 \times 2^{4-3} \times 2^{-4}$$

$$X \times y_2 \times 2^{4-2} \times 2^{-4}$$

$$X \times y_1 \times 2^{4-1} \times 2^{-4}$$

整个运算过程中用到两种操作: 加法 + 左移

因而, 可用ALU和移位器来实现乘法运算



- 数学推导过程：

$$X \times Y = X \times (0.y_1 y_2 \dots y_n)$$

$$= X \times y_1 \times 2^{-1} + X \times y_2 \times 2^{-2} + \dots + X \times y_{n-1} \times 2^{-(n-1)} + X \times y_n \times 2^{-n}$$

$$= \underbrace{2^{-1} (2^{-1} (2^{-1} \dots 2^{-1} (2^{-1} (0 + X \times y_n) + X \times y_{n-1}) + \dots + X \times y_2) + X \times y_1)}_{n \text{ 个 } 2^{-1}}$$

- 上述推导过程具有明显的递归性质，因此，无符号数乘法过程可归结为循环计算下列算式的过程：设 $P_0 = 0$ ，每步的乘积为：

$$P_1 = 2^{-1} (P_0 + X \times y_n)$$

$$P_2 = 2^{-1} (P_1 + X \times y_{n-1})$$

.....

$$P_n = 2^{-1} (P_{n-1} + X \times y_1)$$

**迭代过程从乘数最低位 y_n 和 $P_0=0$ 开始，
经 n 次“判断（ y_i 是否为0）-加法（不为0时）-右移”循环，
直到求出 P_n 为止**

- 其递推公式为： $P_{i+1} = 2^{-1} (P_i + X \times y_{n-i}) \quad (i = 0, 1, 2, 3, \dots, n-1)$
- 最终乘积 $P_n = X \times Y$



- 手工乘法的特点：

- ① 每步计算： $X \times y_i$ ，若 $y_i = 0$ ，则得0；若 $y_i = 1$ ，则得 X
- ② 把①求得的各项结果 $X \times y_i$ 逐次左移，可表示为 $X \times y_i \times 2^{-n} \times 2^{n-i}$
- ③ 对②中结果求和，即 $\sum (X \times y_i \times 2^{-i})$ ，这就是两个无符号数的乘积

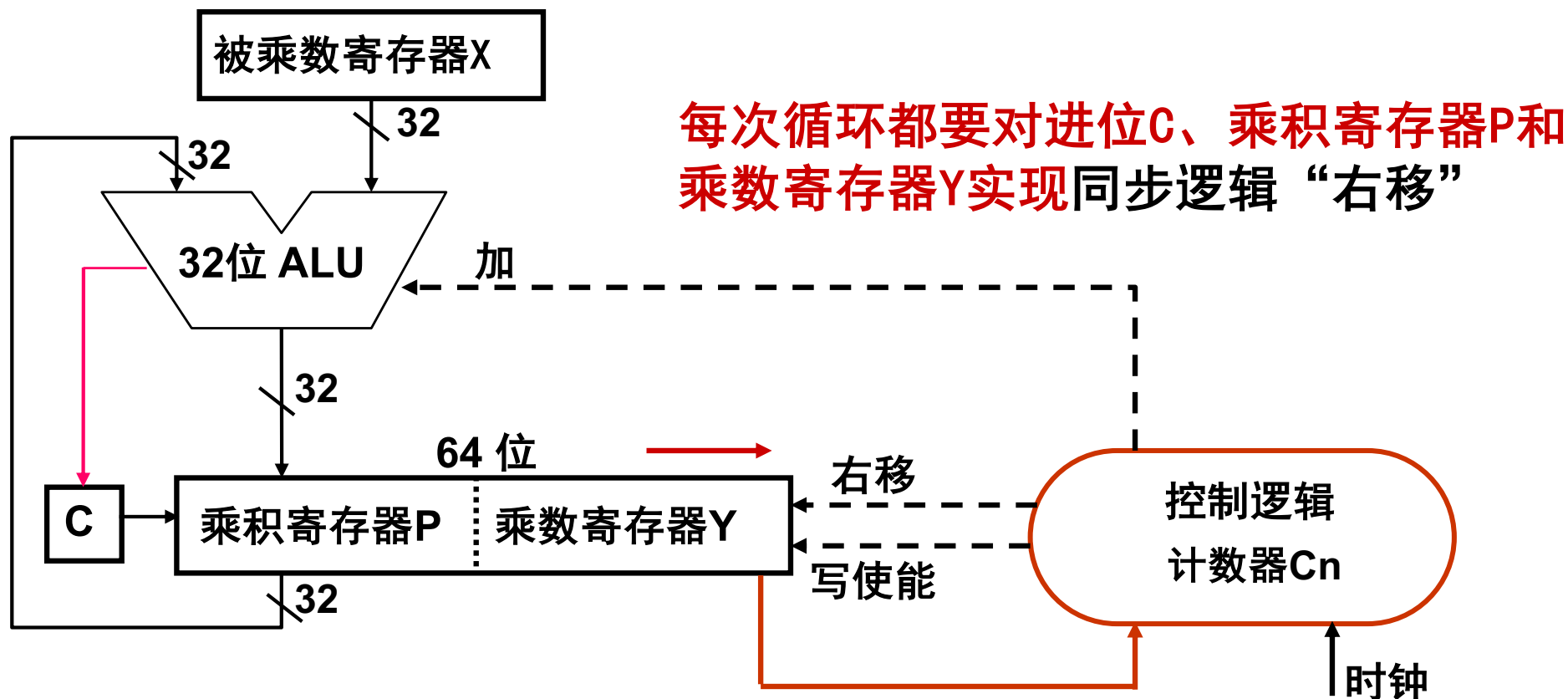
- 计算机内部稍作以下改进：

- ① 每次得 $X \times y_i$ 后，与前面所得结果累加，得到 P_i ，称之为部分积。因为不等到最后一次求和，减少了保存各次相乘结果 $X \times y_i$ 的开销。
- ② 每次得 $X \times y_i$ 后，不将它左移与前次部分积 P_i 相加，而将部分积 P_i 右移后与 $X \times y_i$ 相加。因为加法运算始终对部分积中高 n 位进行。故用 n 位加法器可实现二个 n 位数相乘。
- ③ 对乘数中为“1”的位执行加 X 和右移，对为“0”的位只执行右移，而不执行加法运算。



32位乘法运算的硬件实现

计算机组成原理



- 被乘数寄存器X：存放被乘数
- 乘积寄存器P：开始置初始部分积 $P_0 = 0$ ；结束时，存放的是64位乘积的高32位
- 乘数寄存器Y：开始时置乘数；结束时，存放的是64位乘积的低32位
- 进位触发器C：保存加法器的进位信号
- 循环次数计数器Cn：存放循环次数。初值32，每循环一次，Cn减1，Cn=0时结束
- ALU：乘法核心部件。在控制逻辑控制下，对P和X的内容“加”，在“写使能”控制下运算结果被送回P，进位位在C中



Example: 无符号整数乘法运算

计算机组成原理

举例说明：若需计算 $z=x*y$ ； x 、 y 和 z 都是unsigned类型。

设 $x=1110$ $y=1101$ 应用递推公式： $P_{i+1}=2^{-1}(x*y_{n-i}+ P_i)$

C	乘积P	乘数Y	
0	0000	1101	P_0
+	1110		
0	1110	1101	
→ 0	0111	0110	P_1
→ 0	0011	1011	P_2
+	1110		
1	0001	1011	
→ 0	1000	1101	P_3
+	1110		
1	0110	1101	
→ 0	1011	0110	P_4

可用一个双倍字长的乘积寄存器；也可用两个单倍字长的寄存器。

部分积初始为0。

保留进位位。

右移时进位、部分积和剩余乘数一起进行逻辑右移。

验证： $x=14$, $y=13$, $z=x*y=182$

当 z 取4位时，结果发生溢出，因为高4位不为全0！



串行乘法器的优劣分析

优：不需要很多器件，硬件结构简单

劣：速度太慢，执行一次乘法操作的时间约是加法操作的 n 倍

- 由于乘法操作大约占全部算术运算的 $1/3$ ，故采用高速乘法部件是非常必要的。
- 自从大规模集成电路问世以来，各种形式的高速单元阵列乘法器应运而生，它们都属于并行乘法器。



实现这个乘法过程所需要的操作和我们习惯的方法类似：

$$\begin{array}{r}
 \begin{array}{cccccc}
 & & & a_{m-1} & a_{m-2} & \cdots & a_1 & a_0 \\
 \times) & & & & b_{n-1} & \cdots & b_1 & b_0 \\
 \hline
 & & & a_{m-1}b_0 & a_{m-2}b_0 & \cdots & a_1b_0 & a_0b_0 \\
 & & a_{m-1}b_1 & a_{m-2}b_1 & \cdots & a_1b_1 & a_0b_1 & \\
 & & & \cdot & & & \cdot & \\
 & & & & \cdot & & & \cdot \\
 & & & & & & & \cdot \\
 +) & a_{m-1}b_{n-1} & a_{m-2}b_{n-1} & \cdots & a_1b_{n-1} & a_0b_{n-1} & & \\
 \hline
 p_{m+n-1} & p_{m+n-2} & p_{m+n-3} & \cdots & p_{n-1} & \cdots & p_1 & p_0
 \end{array}
 \end{array}$$



设有两个不带符号的二进制整数 $A = a_{m-1} \dots a_1 a_0$,
 $B = b_{n-1} \dots b_1 b_0$, 它们的数值分别为 a 和 b , 即:

$$a = \sum_{i=0}^{m-1} a_i 2^i \quad b = \sum_{j=0}^{n-1} b_j 2^j$$

在二进制乘法中, 被乘数 A 与乘数 B 相乘, 产生 $m+n$ 位乘积 P , $P = p_{m+n-1} \dots p_1 p_0$, 则乘积 P 的数值为:

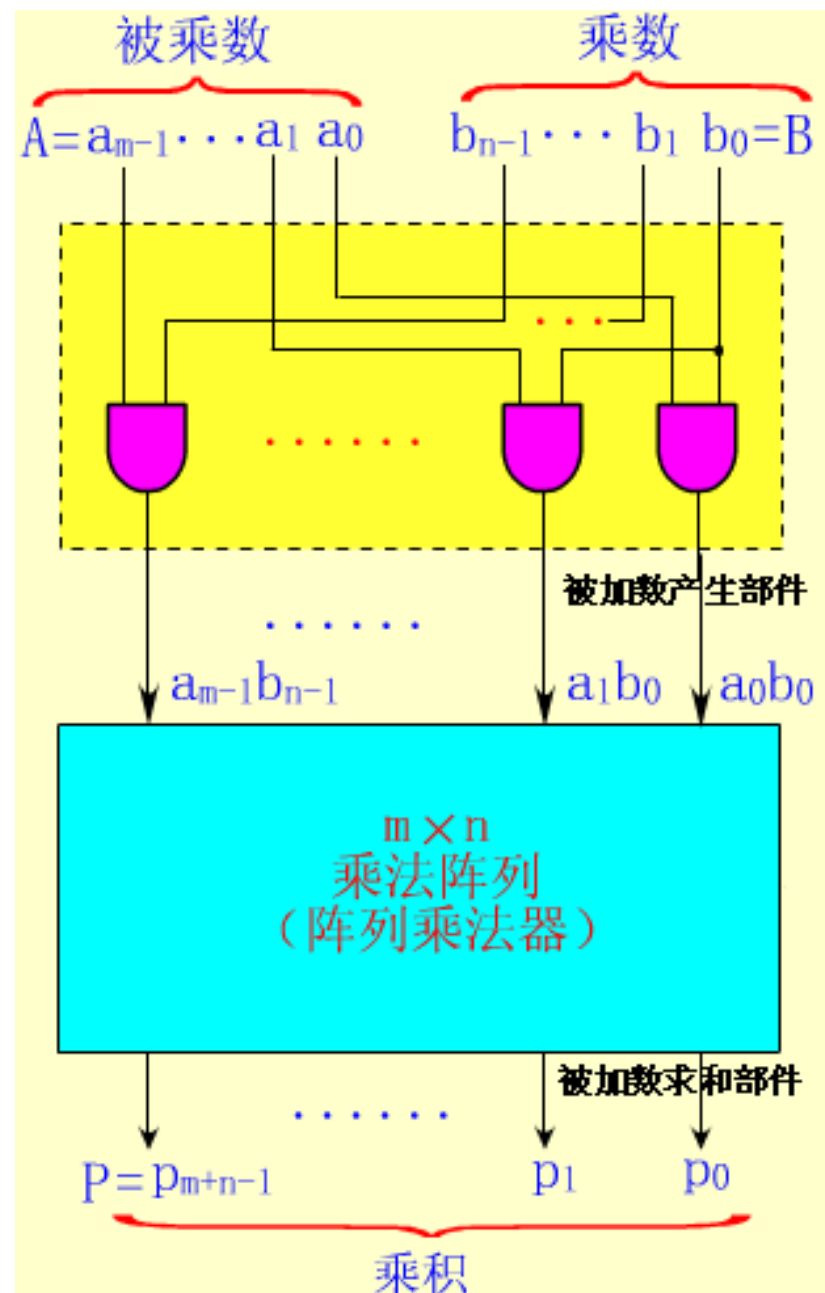
$$p = ab = \left(\sum_{i=0}^{m-1} a_i 2^i \right) \left(\sum_{j=0}^{n-1} b_j 2^j \right) = \sum_{i=0}^{m-1} \sum_{j=0}^{n-1} (a_i b_j) 2^{i+j} = \sum_{k=0}^{m+n-1} p_k 2^k$$



每一个部分乘积项（位积）

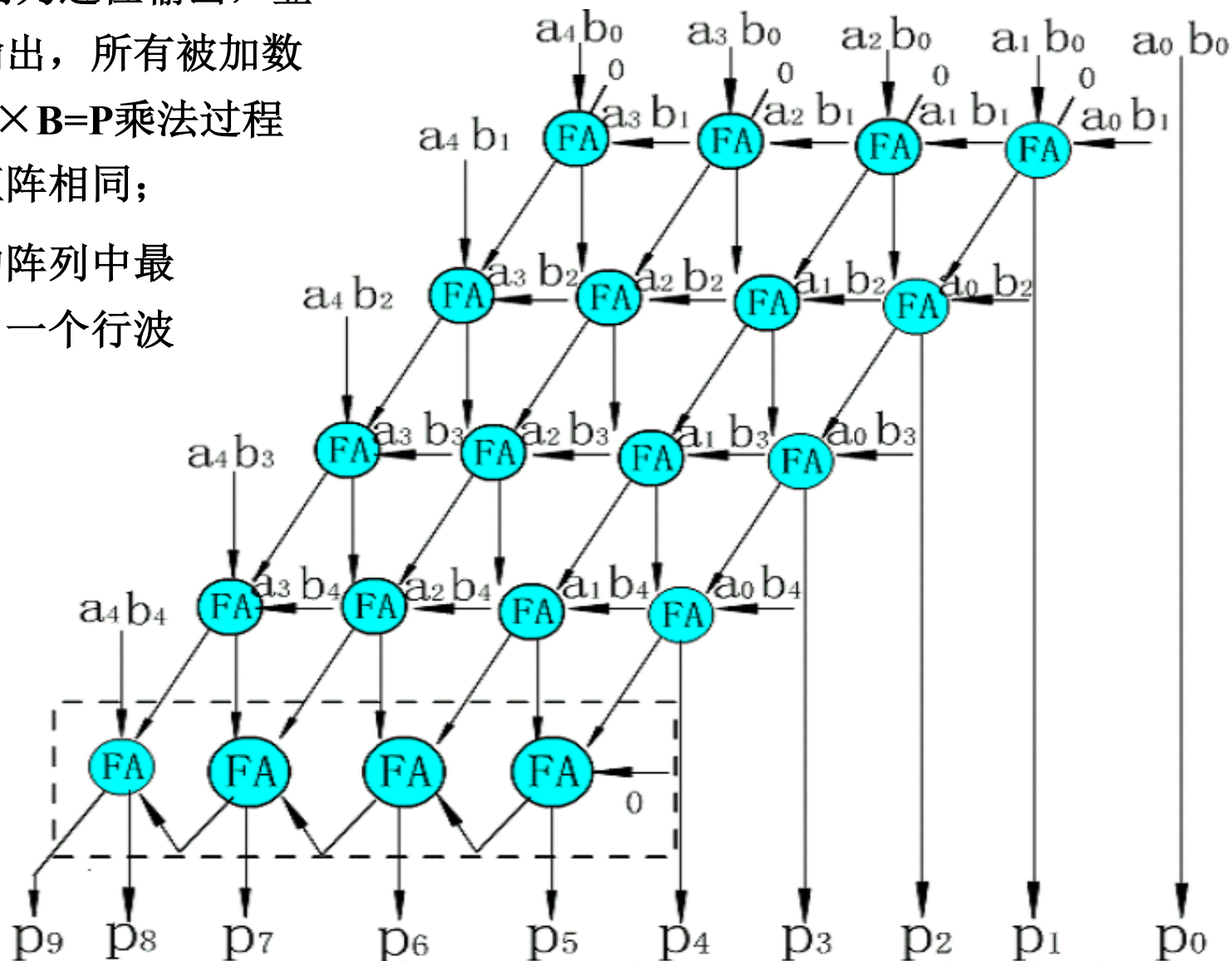
$a_i b_j$ 叫做一个被加数，这 $m \times n$ 个被加数可以用 $m \times n$ 个“与”门并行产生。

下面以5位乘5位不带符号的阵列乘法器（ $m=n=5$ ）为例来说明并行阵列乘法器的基本原理。





- FA的斜线方向为进位输出，竖线方向为和输出，所有被加数项的排列和 $A \times B = P$ 乘法过程中的被加数矩阵相同；
- 用虚线围住的阵列中最后一行构成了一个行波进位加法器。





原码乘法算法

- 用于浮点数尾数乘运算
- 符号与数值分开处理：积符异或得到，数值用无符号乘法运算

例：设 $[x]_{\text{原}}=0\cdots 1110$ ， $[y]_{\text{原}}=1\cdots 1101$ ，计算 $[x \times y]_{\text{原}}$

解：数值部分用无符号数乘法算法计算： $1110 \times 1101 = 1011\ 0110$

符号位： $0 \oplus 1 = 1$ ，所以： $[x \times y]_{\text{原}} = 1.10110110$



- 串行乘法器：速度慢
- 阵列乘法器：硬件规模大，需要大量加法器
- Booth乘法器
 - 观察
 - $13 * 15 = 13 * 1111$ ，需要4次串行加法
 - $13 * (10000 - 0001)$ ，2次加法1次减法
 - 利用分组编码减少加减法次数，常用于有符号补码乘法
 - 乘数最低位补0，一次考察2位
 - 00、11：仅算术右移
 - 01：加被乘数，算术右移
 - 10：减被乘数，算术右移



布斯算法举例

已知 $[X]_{\text{补}} = 1101$, $[Y]_{\text{补}} = 0110$, 计算 $[X \times Y]_{\text{补}}$ $[-X]_{\text{补}} = 0011$

$X = -3$, $Y = 6$, $X \times Y = -18$, $[X \times Y]_{\text{补}}$ 应等于 11101110 或结果溢出

P	Y	y_{-1}	说明
0000	0110 <u>0</u>		设 $y_{-1} = 0$, $[P_0]_{\text{补}} = 0$
		$\rightarrow 1$	$y_0 y_{-1} = 00$, P、Y 直接右移一位
0000	0011 <u>0</u>		得 $[P_1]_{\text{补}}$
+0011			$y_1 y_0 = 10$, $+[-X]_{\text{补}}$
0011		$\rightarrow 1$	P、Y 同时右移一位
0001	1001 <u>1</u>		得 $[P_2]_{\text{补}}$
		$\rightarrow 1$	$y_2 y_1 = 11$, P、Y 直接右移一位
0000	1100 <u>1</u>		得 $[P_3]_{\text{补}}$
+1101			$y_3 y_2 = 01$, $+ [X]_{\text{补}}$
1101		$\rightarrow 1$	P、Y 同时右移一位
1110	1110	0	得 $[P_4]_{\text{补}}$



1. 手算运算步骤

例：设被除数 $x=0.1001$ ，除数 $y=0.1011$ ，仿十进制除法运算，手算求 $x \div y$ 的过程。

	<u>0.1 1 0 1</u>	商 q	
0.1 0 1 1	) 0.1 0 0 1 0	$x(r_0)$	被除数小于除数，商0
	— 0.0 1 0 1 1	$2^{-1} y$	除数右移1位,减除数, 商1
	0.0 0 1 1 1 0	r_1	得余数 r_1
	— 0.0 0 1 0 1 1	$2^{-2} y$	除数右移1位,减除数, 商1
	0.0 0 0 0 1 1 0	r_2	得余数 r_2
	— 0.0 0 0 1 0 1 1	$2^{-3} y$	除数右移1位,不减除数,商0
	0.0 0 0 0 1 1 0 0	r_3	得余数 r_3
	— 0.0 0 0 0 1 0 1 1	$2^{-4} y$	除数右移1位,减除数, 商1
	— 0.0 0 0 0 0 0 0 1	r_4	得余数 r_4

得 $x \div y$ 的商 $q=0.1101$ ，余数为 $r=0.00000001$ 。



机器运算与手算的不同

1) 在计算机中，小数点是固定的，不能简单地采用手算的办法。为便于机器操作，除数 Y 固定不变，被除数和余数进行左移（相当于乘2）。

2) 机器不会心算，必须先作减法：

若余数为正，才知道够减；

若余数为负，才知道不够减。不够减时必须恢复原来的余数，以便再继续往下运算。这种方法称为恢复余数法。

3) 要恢复原来的余数，只要当前的余数加上除数即可。但由于要恢复余数，使除法进行过程的步数不固定，因此控制比较复杂。



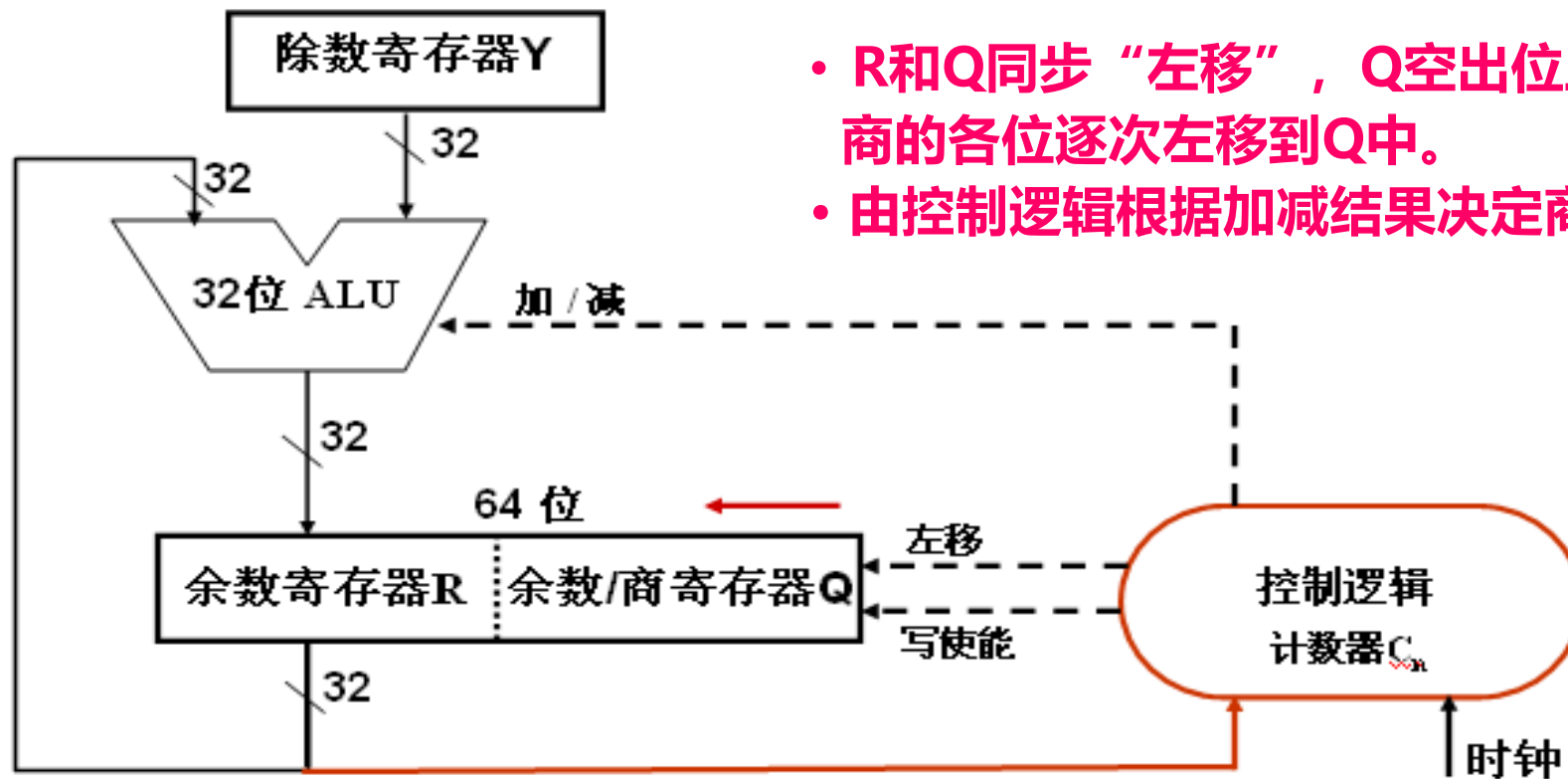
由于恢复余数法的步数不固定，控制比较复杂，实际上常用的是**加减交替法**。

特点：当运算过程中出现不够减的情况，不必恢复余数，而是根据余数的符号，继续往下运算，因此步数固定，控制简单。

运算规则：

当余数为正时，商**1**，余数左移一位，减除数；

当余数为负时，商**0**，余数左移一位，加除数。



- R和Q同步“左移”，Q空出位上“商”，商的各位逐次左移到Q中。
- 由控制逻辑根据加减结果决定商为0还是1

- 除数寄存器Y：存放除数。
- 余数寄存器R：初始时高位部分为高32位被除数；结束时是余数。
- 余数/商寄存器Q：初始时为低32位被除数；结束时是32位商。
- 循环次数计数器 C_n ：存放循环次数。初值是32，每循环一次， C_n 减1，当 $C_n=0$ 时，除法运算结束。
- ALU：除法核心部件。在控制逻辑控制下，对于寄存器R和Y的内容进行“加/减”运算，在“写使能”控制下运算结果被送回寄存器R。



- 原码除法

- 商符和商值分开处理
 - 商的数值部分由无符号数除法求得
 - 商符由被除数和除数的符号确定：同号为0，异号为 1
- 余数的符号同被除数的符号

- 补码除法

- 方法1：同原码除法一样，先转换为正数，先用无符号数除法，然后修正商和余数。
- 方法2：直接用补码除法，符号和数值一起进行运算，商符直接在运算中产生。

若是两个 n 位补码整数除法运算，则被除数进行符号扩展。

若被除数为 $2n$ 位，除数为 n 位，则被除数无需扩展。



不恢复余数除法(加减交替法)

计算机组成原理

验证: $7 / 2 = 3$ 余 1 $-D = 1110$

R: 被除数 (中间余数) ; D: 除数

	R: 0000 0111
R = R-D	1110
	1110
sl R, 0	R: 1100 1110
R = R+D	0010
	1110
sl R, 0	R: 1101 1100
R = R+D	0010
	1111
sl R, 0	R: 1111 1000
R = R+D	0010
	0001
sl R, 0	R: 0011 0001
R = R-D	1110
	0001 0011

第0步 “试商”

不恢复余数法、加减交替法
负, 0, 加
正, 1, 减

第1位商为0, 表示结果
不溢出, 最后左移出去
, 并加上最后一位商

逻辑设计和硬件控制语言HCL

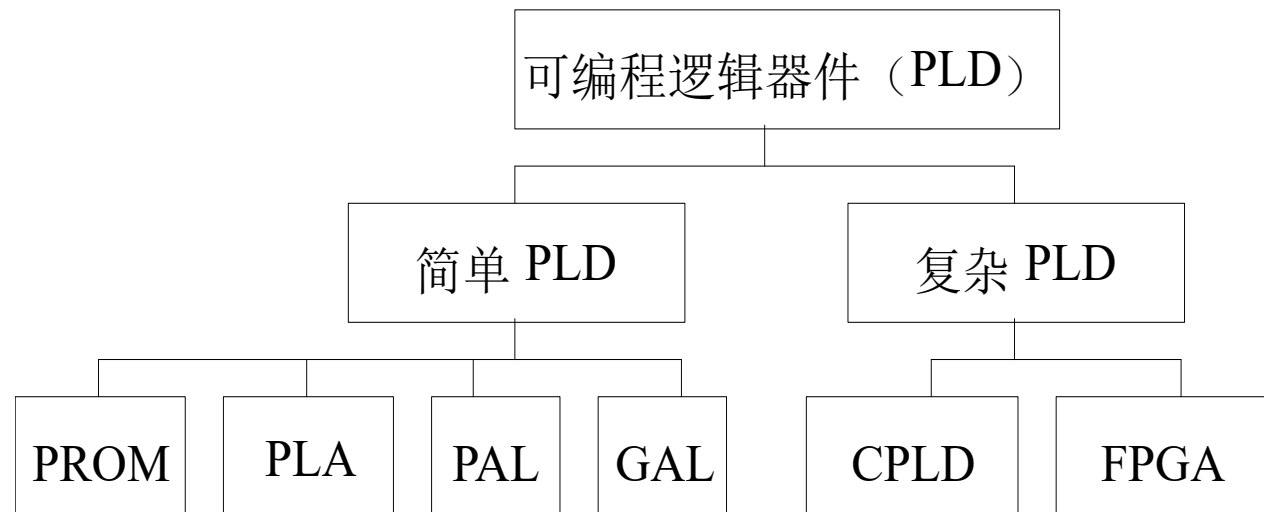
- 逻辑门
 - 数字电路的基本计算单元
 - 输出等于输入位值的某个布尔函数
- 固定逻辑标准芯片
 - 硬件设计者通过逻辑电路图设计（纸笔、图形终端）
 - 功能单一，不能随需求改变
- 可编程逻辑器件PLD
 - 一种用于实现逻辑电路的通用器件
 - 包含多个逻辑单元，编程控制开关链接，从而构成不同功能的逻辑电路
 - 结构主要由与阵列和或阵列构成

HCL

可编程逻辑器件PLD

- 分类

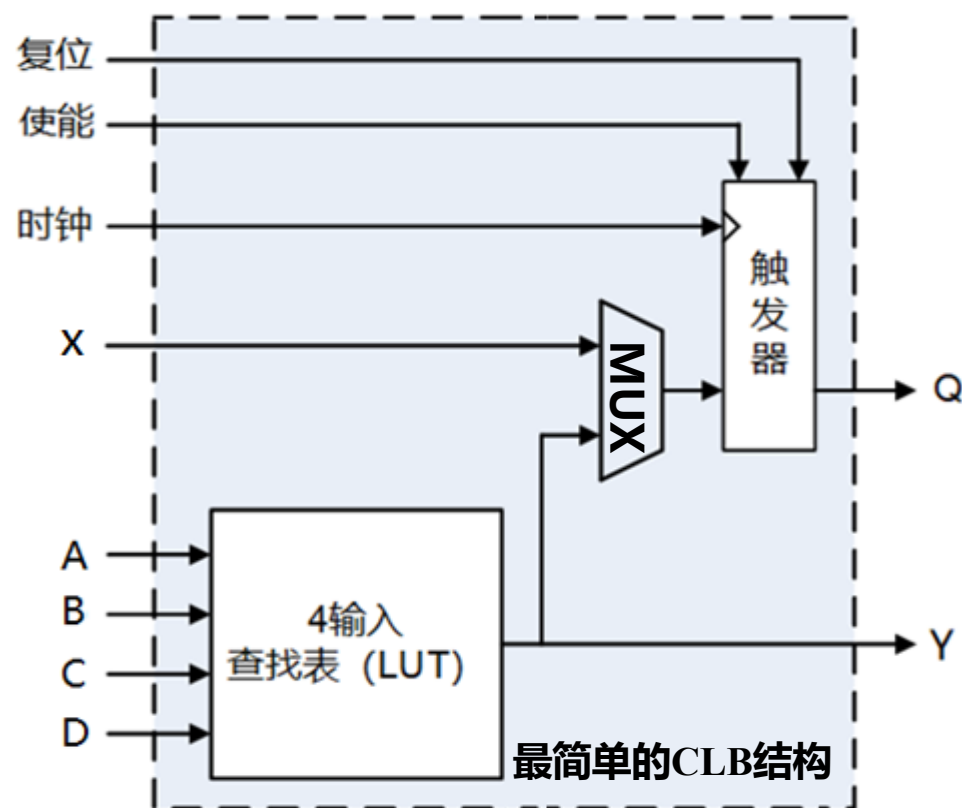
- 简单PLD：逻辑门数量在500以下
- 复杂PLD：逻辑门数量超过500，集成度高



FPGA

通过HDL描述逻辑电路后，EDA工具自动计算逻辑电路
所有可能的结果，并以真值表形式事先写入RAM中

- 现场可编程门阵列（Field Programmable Gate Array, FPGA）
 - 一种高集成度的复杂可编程逻辑器件，可通过EDA（Electronic Design Automation）软件对其进行配置和编程，可反复擦写
 - 内部包含大量可配置逻辑块CLB，它由若干查找表（Look-Up Table, LUT）及多路选择器、进位链、触发器FF等附加逻辑组成



硬件控制语言HCL

- 一种用形式化方法来描述数字电路和设计数字逻辑系统的语言
 - 类似编程语言，但描述硬件结构，而非程序行为
 - VHDL，1981年由美国军方组织开发，语法类Ada
 - Verilog，1984年Gateway Design Automation公司于作为一个逻辑模拟的语言开发，语法类C
- 共同的特点
 - 能形式化地抽象表示电路的结构和行为；
 - 支持逻辑设计中层次与领域的描述；
 - 可借用高级语言的精巧结构来简化电路行为的描述；
 - 具有电路仿真与验证机制以保证设计的正确性；
 - 支持电路描述由高层到低层的综合转换；
 - 硬件描述与实现工艺无关；
 - 便于文档管理；
 - 易于理解 and 设计重用。

HCL

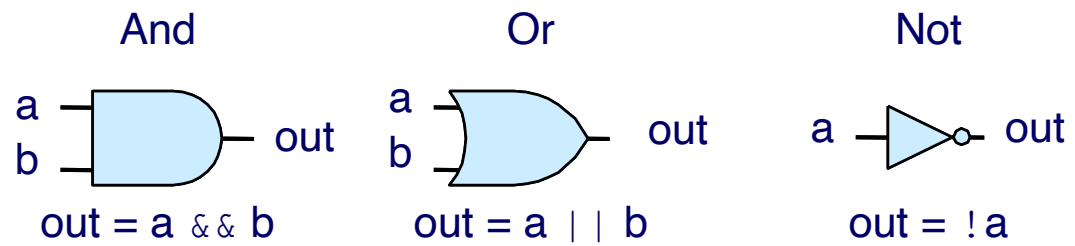
- Very simple hardware description language
 - Boolean operations have syntax similar to C logical operations
- Can only express limited aspects of hardware operation
 - Parts we want to explore and modify
- Use it to describe control logic for processors
- 数据类型
 - `bool: Boolean`
 - `a, b, c, ...`
 - `int: words`
 - `A, B, C, ...`
 - Does not specify word size---4-64-bit words, ...
- 表达式
 - `bool a = bool-expr ;`
 - `int A = int-expr ;`

HCL

- Classify by type of value returned
- 逻辑门组成组合电路，逻辑门的选择和组合电路的构建方式由HCL表达式给出
- 布尔表达式
 - 逻辑门
 - `a && b, a || b, !a`
 - 字比较
 - `A == B, A != B, A < B, A <= B, A >= B, A > B`
 - 集合关系
 - `A in { B, C, D }`
 - Same as `A == B || A == C || A == D`
- 整数表达式
 - Case expressions（情况表达式）
 - `[ a : A; b : B; c : C ]` 顺序求值，第一个求值为1的情况被选中
 - Evaluate test expressions a, b, c, ... in sequence
 - Return word expression A, B, C, ... for first successful test

HCL

- 逻辑门

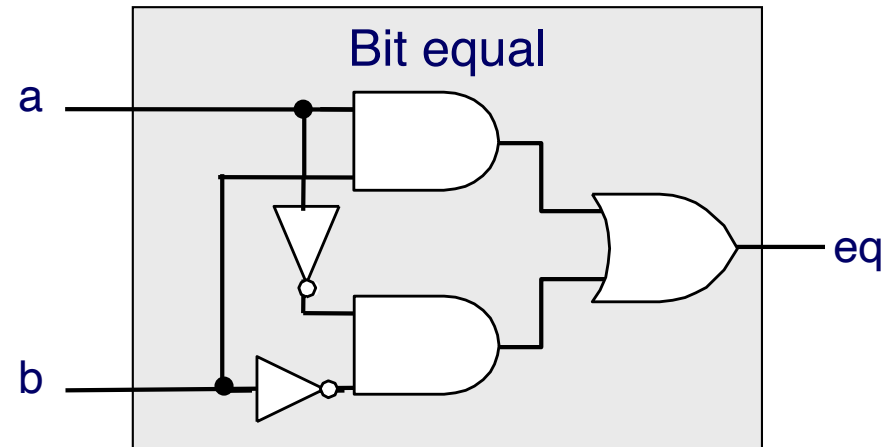


HCL

- 位相等

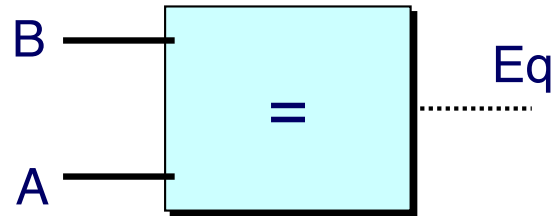
- (a: bool, b: bool) -> eq: bool
 - (1, 1)->1, (0, 0)-> 1
 - others->0

```
bool eq = (a&&b) || (!a&&!b)
```

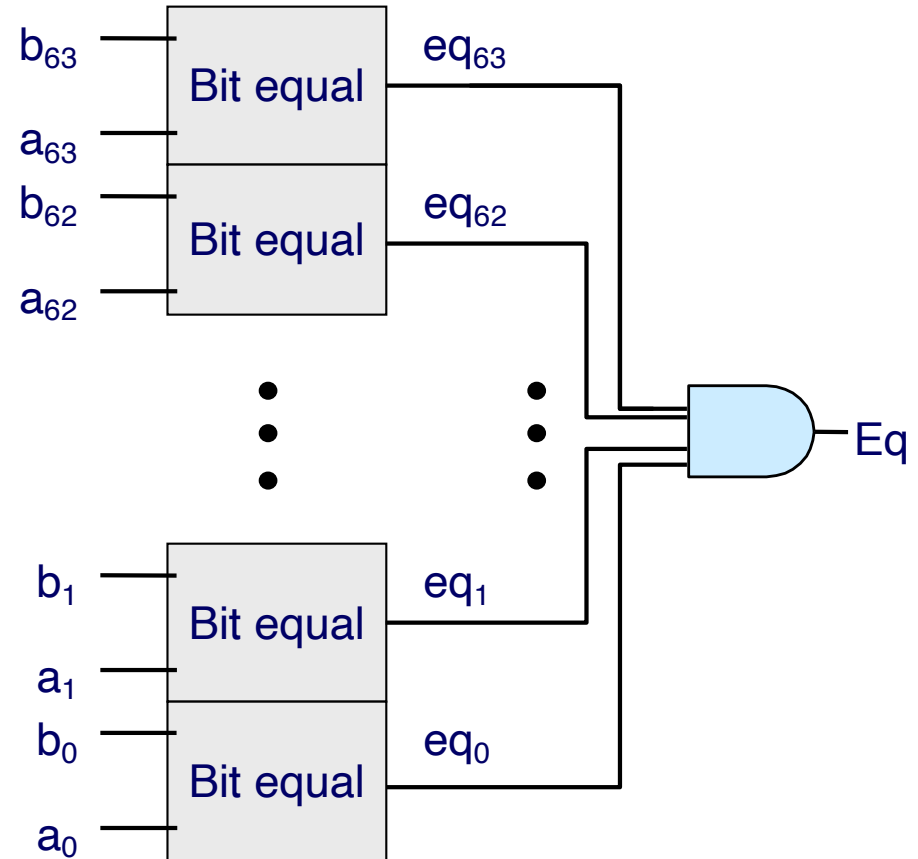


HCL

- 字相等
 - $(A: \text{int}, B: \text{int}) \rightarrow \text{Eq}: \text{bool}$
 - 64位字长



```
bool Eq = (A == B)
```

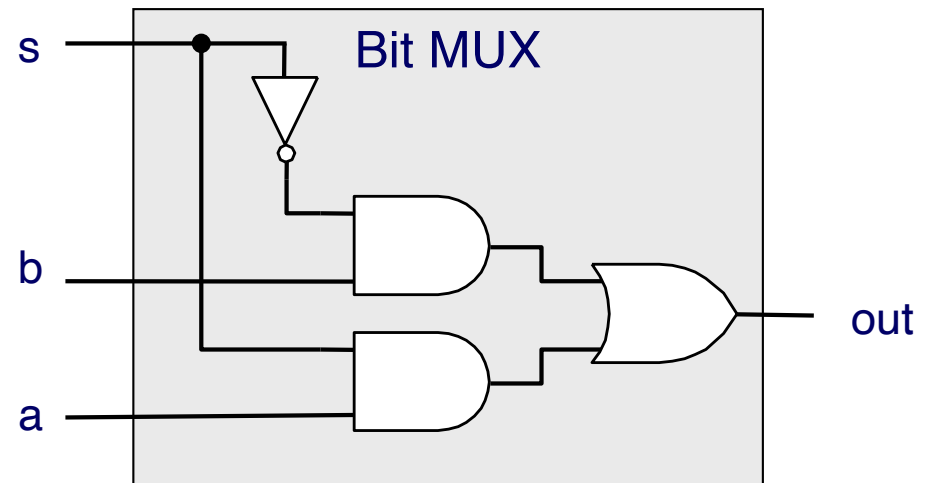


HCL

- 位多路复用器MUX

- 控制信号s, 数据输入a、b
- $(s, a, b) \rightarrow out$
- $s=1, out=a$
- $s=0, out=b$

```
bool out = (s&&a) || (!s&&b)
```

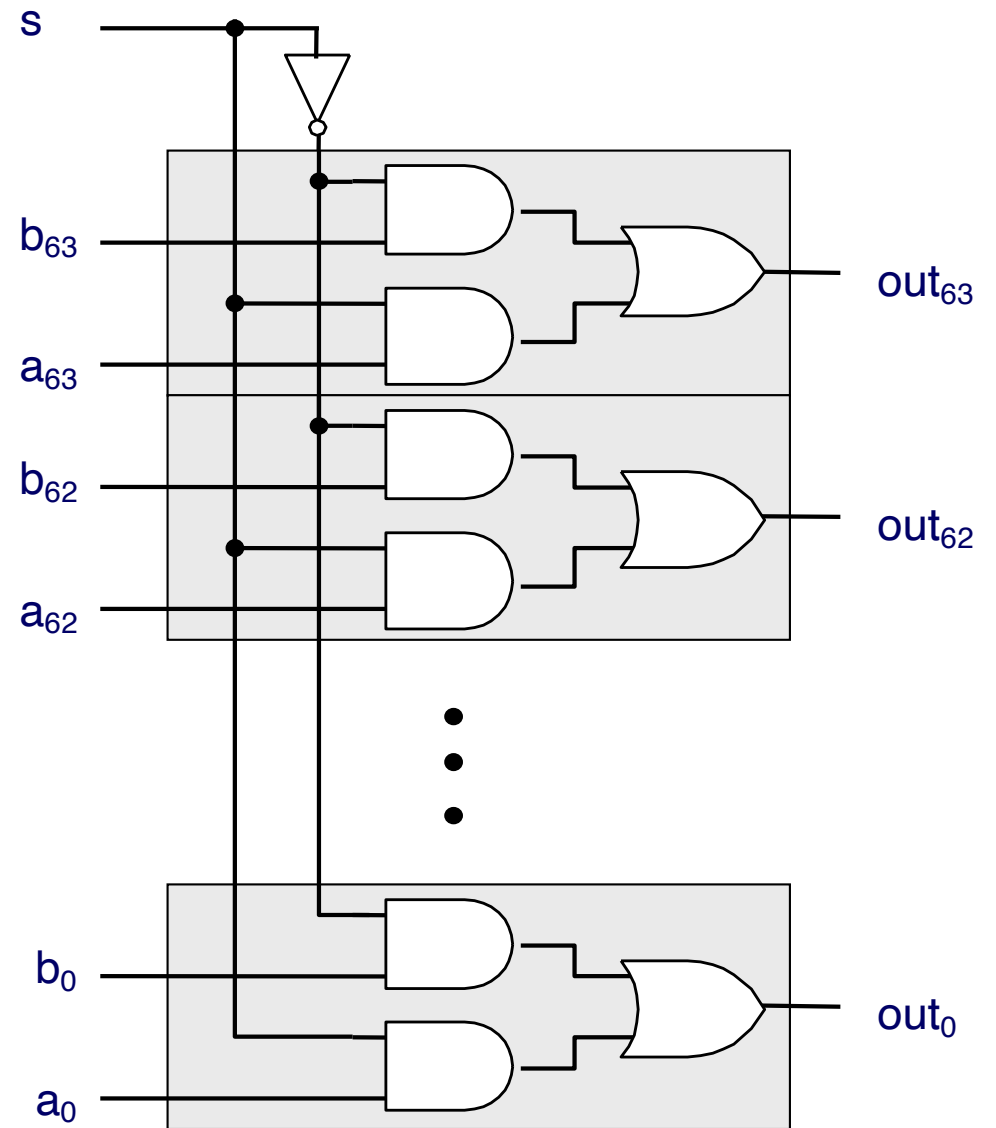
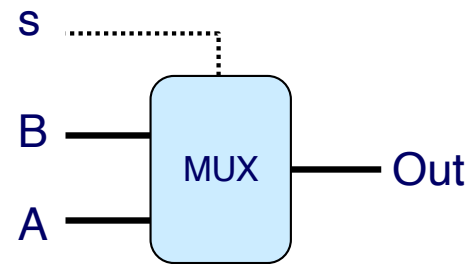


HCL

- 字MUX

- 根据s选择A或B
- 64位字长
- 64个位MUX, 只产生一次!s

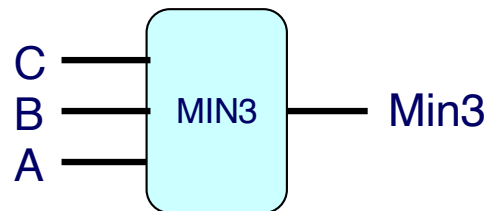
```
int Out = [  
    s : A;  
    1 : B;  
];
```



HCL

- **Minimum of 3 Words**

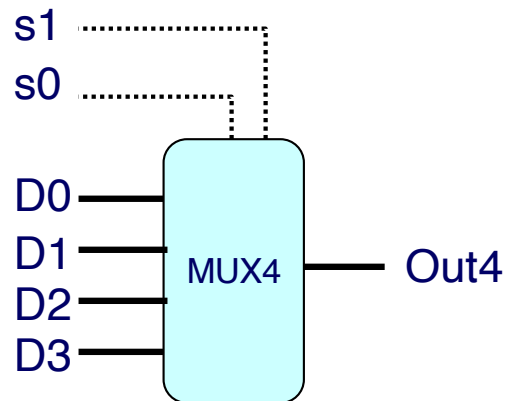
- Find minimum of three input words
- HCL case expression
- Final case guarantees match



```
int Min3 = [  
    A < B && A < C : A;  
    B < A && B < C : B;  
    1                : C;  
];
```

HCL

- 4-Way Multiplexor (四路复用器)
 - Select one of 4 inputs based on 2 control bits
 - HCL case expression
 - Simplify tests by assuming sequential matching

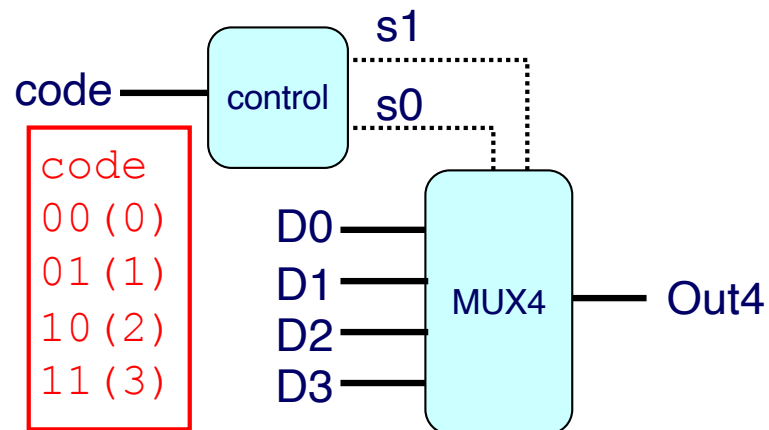


```
int Out4 = [  
    !s1&&!s0: D0; #00  
    !s1      : D1; #01  
    !s0      : D2; #10  
    1        : D3; #11  
];
```

HCL

- 集合关系

- $\text{lexpr} \in \{\text{inexpr}_1, \text{inexpr}_2, \dots, \text{inexpr}_k\}$
- E.g., 检测某一个指令是否属于某一类指令
- 用一个两位code产生四路复用器的控制，选择4个数据字



```
bool s1 = code==2 || code==3;  
bool s0 = code==1 || code==3;
```



```
bool s1 = code in {2, 3};  
bool s0 = code in {1, 3};
```